Experiment No. 6

Aim: To write a program to implement the Travelling Salesman Problem (TSP).

Theory:

The Travelling Salesman Problem (TSP) is a classic combinatorial optimization problem in computer science and operations research. Given a list of cities and the distances between each pair, the goal is to find the shortest possible route that visits every city exactly once and returns to the starting city.

TSP is an NP-hard problem, meaning no efficient polynomial-time solution is known for large inputs. For small inputs, the brute-force approach of trying all permutations is feasible. The total number of possible routes for n cities is (n-1)! / 2, which grows very quickly.

TSP has real-world applications in logistics, supply chain management, circuit board drilling, and DNA sequencing.

Algorithm:

- Generate all possible permutations of cities (excluding the starting city).
- For each permutation, compute the total travel cost of the route.
- Track the permutation with the minimum total cost.
- Return the optimal path and its minimum cost.

Complexity:

- **Time Complexity:** O(n!) for the brute-force approach, where n is the number of cities.
- **Space Complexity:** O(n) for storing the current path.

Program:

```
from itertools import permutations

def tsp(graph, start):
    nodes = list(graph.keys())
    nodes.remove(start)
    min_cost = float('inf')
    best_path = []

    for perm in permutations(nodes):
```

```
            path = [start] + list(perm) + [start]
            cost = 0
            valid = True
            for i in range(len(path) - 1):
                if path[i+1] in graph[path[i]]:
                    cost += graph[path[i]][path[i+1]]
                else:
                    valid = False
                    break
            if valid and cost < min_cost:
                min_cost = cost
                best_path = path

        return best_path, min_cost

graph = {
    'A': {'B': 10, 'C': 15, 'D': 20},
    'B': {'A': 10, 'C': 35, 'D': 25},
    'C': {'A': 15, 'B': 35, 'D': 30},
    'D': {'A': 20, 'B': 25, 'C': 30}
}

path, cost = tsp(graph, 'A')
print('Optimal Path:', ' -> '.join(path))
print('Minimum Cost:', cost)
```

Output:

```
Optimal Path: A -> B -> D -> C -> A
Minimum Cost: 80
```

Conclusion:

The Travelling Salesman Problem was successfully implemented using the brute-force permutation approach. The program correctly identifies the shortest route visiting all cities exactly once and returning to the start. While this approach is feasible for small inputs, heuristic algorithms like nearest neighbour or dynamic programming are preferred for larger inputs.

Experiment No. 7

Aim: To write a program to implement Hill Climbing and Steepest Ascent Hill Climbing algorithm.

Theory:

Hill Climbing is a local search algorithm used in optimization problems. It starts from an initial solution and iteratively moves to a neighboring solution that improves the objective function. It continues until no better neighbor is found, at which point it has reached a local maximum (or minimum, depending on the problem).

Simple Hill Climbing selects any neighbor that improves the current solution and moves to it immediately. Steepest Ascent Hill Climbing, on the other hand, evaluates all neighbors and moves to the best one among them. This makes it slower per iteration but potentially more accurate in reaching better solutions.

A major limitation of hill climbing is that it can get stuck at local optima, ridges, or plateaus. Techniques like random restarts and simulated annealing are used to overcome this.

Algorithm:

- Start with an initial solution (current state).
- Evaluate all neighboring states of the current state.
- Simple HC: Move to the first neighbor that is better. Steepest Ascent HC: Move to the best neighbor overall.
- If no neighbor is better, stop and return the current state as the solution.
- Repeat until termination condition is met.

Complexity:

- **Time Complexity:** O(iterations x neighbors) per run.
- **Space Complexity:** O(1) — only the current state is stored.

Program:

```
import random

def objective(x):
    return -(x**2) + 4*x + 6   # maximise this function
```

```
# Simple Hill Climbing
def hill_climbing(start, step=0.1, iterations=1000):
    current = start
    for _ in range(iterations):
        neighbor = current + random.choice([-step, step])
        if objective(neighbor) > objective(current):
            current = neighbor
    return current, objective(current)

# Steepest Ascent Hill Climbing
def steepest_ascent(start, step=0.1, neighbors=10):
    current = start
    while True:
        candidates = [current + (i - neighbors//2) * step
                      for i in range(neighbors)]
        best = max(candidates, key=objective)
        if objective(best) <= objective(current):
            break
        current = best
    return current, objective(current)

start = 0.0
hc_x, hc_val   = hill_climbing(start)
sa_x, sa_val   = steepest_ascent(start)

print(f'Hill Climbing       -> x = {hc_x:.4f}, f(x) = {hc_val:.4f}')
print(f'Steepest Ascent HC  -> x = {sa_x:.4f}, f(x) = {sa_val:.4f}')
```

Output:

```
Hill Climbing       -> x = 2.0000, f(x) = 10.0000
Steepest Ascent HC  -> x = 2.0000, f(x) = 10.0000
```

Conclusion:

Both Hill Climbing and Steepest Ascent Hill Climbing were successfully implemented. The algorithms correctly maximized the objective function. Steepest Ascent HC evaluates all neighbors before making a move, making it more systematic than simple Hill Climbing. Both methods are susceptible to local optima but work well for unimodal functions.

Experiment No. 8

Aim: To write a program to load data in a Google Colab Notebook from a CSV file.

Theory:

Google Colab is a free cloud-based platform that allows running Python code in a Jupyter notebook environment directly in the browser. It provides access to GPUs and has most data science libraries pre-installed, making it ideal for AI and machine learning experiments.

Loading data from a CSV (Comma-Separated Values) file is one of the most fundamental steps in any data analysis project. In Google Colab, a CSV file can be loaded in multiple ways — by uploading from the local system using the files.upload() method, by mounting Google Drive, or by reading directly from a URL.

Once loaded, the data is read into a pandas DataFrame, which is a two-dimensional tabular structure that allows easy manipulation, filtering, and analysis of data.

Steps to Load CSV in Colab:

- Open Google Colab at https://colab.research.google.com and create a new notebook.
- Use the files.upload() function from the google.colab module to upload a CSV file from your local machine.
- Read the uploaded file into a pandas DataFrame using pd.read_csv().
- Explore the data using head(), shape, dtypes, and describe() functions.

Complexity:

- **Time Complexity:** O(n) where n is the number of rows in the CSV file.
- **Space Complexity:** O(n x m) where n is rows and m is columns.

Program:

```
# Step 1: Upload CSV file in Google Colab
from google.colab import files
uploaded = files.upload()   # opens file picker in Colab

# Step 2: Load the uploaded CSV using pandas
import pandas as pd
import io
```

```
filename = list(uploaded.keys())[0]
df = pd.read_csv(io.BytesIO(uploaded[filename]))

# Step 3: Explore the data
print('Shape:', df.shape)
print('Column Names:', df.columns.tolist())
print()
print('First 5 rows:')
print(df.head())
print()
print('Data Types:')
print(df.dtypes)
print()
print('Basic Statistics:')
print(df.describe())
```

Output:

```
Shape: (150, 5)
Column Names: ['sepal_length', 'sepal_width', 'petal_length',
'petal_width', 'species']
First 5 rows:
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
```

Conclusion:

The experiment demonstrated how to upload and load a CSV file into Google Colab using pandas. The data was successfully read into a DataFrame and explored using standard pandas commands. This is an essential first step for any data analysis or machine learning task in Colab.

Experiment No. 9

Aim: To write a program to implement various Pandas commands on the Titanic Dataset.

Theory:

Pandas is a powerful open-source Python library for data analysis and manipulation. It provides two primary data structures: Series (one-dimensional) and DataFrame (two-dimensional), which are highly efficient and flexible for working with structured data.

The Titanic dataset is one of the most commonly used datasets in data science. It contains information about passengers aboard the RMS Titanic, including details such as age, gender, class, fare, and whether they survived. It is available directly through the seaborn library in Python.

Common Pandas operations include reading data, exploring its structure, handling missing values, filtering rows, grouping data, sorting, and computing statistics. These operations form the foundation of the Exploratory Data Analysis (EDA) process.

Key Pandas Commands Used:

- df.head() — displays the first 5 rows of the dataset.
- df.shape — returns (rows, columns) of the DataFrame.
- df.dtypes — shows the data type of each column.
- df.isnull().sum() — counts missing values per column.
- df.describe() — gives statistical summary of numeric columns.
- df.value_counts() — counts occurrences of unique values.
- df.groupby() — groups data and applies aggregate functions.
- df.sort_values() — sorts rows by a column.

Complexity:

- **Time Complexity:** O(n) for most operations, O(n log n) for sorting, where n is the number of rows.
- **Space Complexity:** O(n x m) for the full DataFrame in memory.

Program:

```
import pandas as pd

# Load the Titanic dataset (pre-installed in Colab via seaborn)
import seaborn as sns
df = sns.load_dataset('titanic')

print('--- Shape ---')
print(df.shape)

print('--- First 5 rows ---')
print(df.head())

print('--- Data Types ---')
print(df.dtypes)

print('--- Missing Values ---')
print(df.isnull().sum())

print('--- Basic Statistics ---')
print(df.describe())

print('--- Survival Count ---')
print(df['survived'].value_counts())

print('--- Survival by Class ---')
print(df.groupby('pclass')['survived'].mean())

print('--- Average Fare by Class ---')
print(df.groupby('pclass')['fare'].mean())

# Drop missing age values and filter passengers older than 30
df_clean = df.dropna(subset=['age'])
older = df_clean[df_clean['age'] > 30]
print('--- Passengers older than 30 ---')
print(older[['name','age','survived']].head())

# Sort by fare descending
print('--- Top 5 by Fare ---')
print(df.sort_values('fare', ascending=False)[['name','fare']].head())
```

Output:

```
Shape: (891, 15)
Survival Count:
0    549
1    342
Average Fare by Class:
pclass
1    84.15
```

```
2    20.66
3    13.68
```

Conclusion:

Various Pandas commands were successfully applied to the Titanic dataset. The experiment demonstrated how to load, explore, clean, filter, group, and sort data using Pandas. These operations are fundamental to data analysis and form the basis of any machine learning pipeline.

Experiment No. 10

Aim: To write a program to integrate SQL and Python.

Theory:

SQL (Structured Query Language) is the standard language for managing and querying relational databases. Python can be integrated with SQL databases using the sqlite3 module (for local databases) or libraries like SQLAlchemy, PyMySQL, or psycopg2 for external databases.

SQLite is a lightweight, serverless, self-contained relational database engine. The sqlite3 module comes built-in with Python and allows creating in-memory or file-based databases without any additional installation. It is commonly used for learning and testing database concepts.

Combining SQL and Python allows developers to perform complex data queries directly within Python programs, and use the results in further computation, visualization, or machine learning. The pandas library's read_sql_query() function makes it easy to load SQL query results into a DataFrame.

Algorithm:

- Create a connection to the SQLite database using sqlite3.connect().
- Create a table using the SQL CREATE TABLE statement.
- Insert records into the table using INSERT INTO.
- Run SELECT queries to retrieve and filter data.
- Load query results into a pandas DataFrame using pd.read_sql_query().
- Perform aggregation queries using GROUP BY and AVG.
- Close the database connection.

Complexity:

- **Time Complexity:** O(n) for insertions and simple queries; O(n log n) for sorted queries.
- **Space Complexity:** O(n) where n is the number of records stored.

Program:

```
import sqlite3
import pandas as pd

# Step 1: Create an in-memory SQLite database
```

```
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Step 2: Create a table
cursor.execute('''
    CREATE TABLE students (
        id      INTEGER PRIMARY KEY,
        name    TEXT,
        course  TEXT,
        marks   INTEGER
    )
''')

# Step 3: Insert records
data = [
    (1, 'Anshika',  'AI', 92),
    (2, 'Rahul',    'AI', 78),
    (3, 'Priya',    'DS', 85),
    (4, 'Arjun',    'DS', 60),
    (5, 'Sneha',    'AI', 95),
]
cursor.executemany('INSERT INTO students VALUES (?,?,?,?)', data)
conn.commit()

# Step 4: Query with SQL and load into pandas
df = pd.read_sql_query('SELECT * FROM students', conn)
print('All Students:')
print(df)

# Step 5: Filter using SQL
df_ai = pd.read_sql_query(
    "SELECT * FROM students WHERE course='AI' AND marks > 80", conn)
print('AI students with marks > 80:')
print(df_ai)

# Step 6: Aggregate
df_avg = pd.read_sql_query(
    'SELECT course, AVG(marks) as avg_marks FROM students GROUP BY course',
conn)
print('Average marks by course:')
print(df_avg)

conn.close()
```

Output:

```
All Students:
   id     name course  marks
0   1  Anshika     AI     92
1   2    Rahul     AI     78
2   3    Priya     DS     85
```

```
3   4    Arjun     DS     60
4   5    Sneha     AI     95
AI students with marks > 80:
   id     name course  marks
0   1  Anshika     AI     92
1   5    Sneha     AI     95
Average marks by course:
  course  avg_marks
0     AI       88.3
1     DS       72.5
```

Conclusion:

SQL was successfully integrated with Python using the sqlite3 module. The experiment demonstrated how to create a database, insert records, run SQL queries, and load results into a pandas DataFrame. This integration is widely used in data engineering, web development, and data analysis applications.

Experiment No. 11

Aim: To write a program to build a basic Linear Regression Model.

Theory:

Linear Regression is one of the simplest and most widely used machine learning algorithms. It is used for predicting a continuous target variable based on one or more input features. In simple linear regression, there is one input feature (X) and one output (y), and the relationship is modelled as a straight line:

$y = mx + c$

where m is the slope (coefficient) and c is the y-intercept. The goal of linear regression is to find the values of m and c that minimize the difference between the predicted values and the actual values, typically measured using Mean Squared Error (MSE).

Scikit-learn provides an easy-to-use LinearRegression class that handles the training and prediction. The model is evaluated using MSE (lower is better) and R2 Score (closer to 1 is better).

Algorithm:

- Prepare the dataset with input feature X and target y.
- Split the dataset into training and testing sets using train_test_split.
- Create and train the LinearRegression model on the training data.
- Use the trained model to predict values for the test set.
- Evaluate the model using MSE and R2 Score.
- Plot the actual data points and the regression line.

Complexity:

- **Time Complexity:** O(n x d) for training, where n is samples and d is features.
- **Space Complexity:** O(d) for storing model parameters.

Program:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import mean_squared_error, r2_score

# Step 1: Create a simple dataset (Hours studied vs Marks)
data = {
    'Hours': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Marks': [35, 40, 50, 55, 65, 70, 75, 80, 88, 95]
}
df = pd.DataFrame(data)

# Step 2: Prepare features and target
X = df[['Hours']]
y = df['Marks']

# Step 3: Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Step 4: Train the model
model = LinearRegression()
model.fit(X_train, y_train)

# Step 5: Predict
y_pred = model.predict(X_test)

# Step 6: Evaluate
print(f'Intercept : {model.intercept_:.2f}')
print(f'Coefficient: {model.coef_[0]:.2f}')
print(f'MSE        : {mean_squared_error(y_test, y_pred):.2f}')
print(f'R2 Score   : {r2_score(y_test, y_pred):.2f}')

# Step 7: Plot
plt.scatter(X, y, color='blue', label='Actual')
plt.plot(X, model.predict(X), color='red', label='Predicted')
plt.xlabel('Hours Studied')
plt.ylabel('Marks')
plt.title('Linear Regression: Hours vs Marks')
plt.legend()
plt.show()
```

Output:

```
Intercept : 28.50
Coefficient: 6.45
MSE        : 12.30
R2 Score   : 0.97
```

Conclusion:

A basic Linear Regression model was successfully built and evaluated. The model achieved a high R2 score of 0.97, indicating a strong linear relationship between hours studied and marks obtained. The

regression line was plotted to visualize the fit. Linear Regression is a foundational algorithm in machine learning and serves as a baseline for more complex models.

Experiment No. 12

Aim: To write a program to implement data visualisation techniques such as plotting bar chart, histogram, and pie chart using matplotlib.

Theory:

Data visualisation is the graphical representation of data and information. It helps in understanding patterns, trends, and outliers in data that might not be visible from raw numbers alone. Python's matplotlib library is one of the most widely used tools for creating static, interactive, and animated visualisations.

A Bar Chart is used to compare quantities across different categories. Each bar represents a category, and its height represents the value. A Histogram displays the frequency distribution of a continuous dataset, grouping values into bins. A Pie Chart shows proportional data as slices of a circle, with each slice representing a category's share of the total.

These three chart types cover a wide range of everyday data visualisation needs in data science, business analytics, and academic reporting.

Algorithm:

- Import the matplotlib.pyplot and numpy libraries.
- Prepare the data — categories and values for each chart type.
- Use plt.bar() to create a bar chart for categorical comparison.
- Use plt.hist() to create a histogram showing frequency distribution.
- Use plt.pie() to create a pie chart showing proportional breakdown.
- Add titles, labels, and legends to each chart for clarity.
- Call plt.show() to render and display each chart.

Complexity:

- **Time Complexity:** O(n) for all plots, where n is the number of data points.
- **Space Complexity:** O(n) for storing data and rendered figure objects.

Program:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
# ── 1. Bar Chart ──────────────────────────────────────────────
subjects = ['Math', 'Science', 'English', 'History', 'AI']
marks    = [85, 92, 78, 65, 95]

plt.figure(figsize=(7, 4))
plt.bar(subjects, marks, color='steelblue', edgecolor='black')
plt.title('Student Marks by Subject')
plt.xlabel('Subject')
plt.ylabel('Marks')
plt.ylim(0, 100)
plt.tight_layout()
plt.show()

# ── 2. Histogram ──────────────────────────────────────────────
data = np.random.normal(loc=70, scale=10, size=200)

plt.figure(figsize=(7, 4))
plt.hist(data, bins=15, color='coral', edgecolor='black')
plt.title('Distribution of Student Marks')
plt.xlabel('Marks')
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# ── 3. Pie Chart ──────────────────────────────────────────────
categories = ['Passed', 'Failed', 'Absent']
values     = [70, 20, 10]
colors     = ['#4CAF50', '#F44336', '#FF9800']

plt.figure(figsize=(6, 6))
plt.pie(values, labels=categories, colors=colors,
        autopct='%1.1f%%', startangle=90)
plt.title('Exam Result Distribution')
plt.axis('equal')
plt.tight_layout()
plt.show()
```

Output:

Three separate plot windows are displayed:

1. Bar Chart — Student Marks by Subject

2. Histogram — Distribution of Student Marks (bell-curve shape)

3. Pie Chart — Exam Result Distribution (70% Passed, 20% Failed, 10% Absent)

Conclusion:

Data visualisation techniques including bar chart, histogram, and pie chart were successfully implemented using matplotlib. Each chart effectively communicates a different aspect of the data — comparison,

distribution, and proportion respectively. Matplotlib provides flexible and powerful tools for visualising data, which is an essential skill in data analysis and machine learning.